

Algorithms for Data Science

Cheatsheet for Exercises 4

Howto: Backtracking → Memoization → Dynamic Programming

In the following, I explain in four steps on how to come up with a fast and space-efficient dynamic programming solution to many problems. At the end, I give some further notes.

Step 1: Backtracking Write a recursive algorithm using the divide and conquer technique (if possible).

```
MainAlgo(a)
├   n = size of a
└   return Algo(a, n)

Algo(a, k)                                // k describes a subproblem of a
├   if k is small then                      // Base case
│   │   Compute the return value directly without recursion.
│   └   return value
├   else
│   │   Make recursive calls on smaller values of k, combine the results, and compute the return value
│   └   return value
```

Step 2: Memoization Store the return values to avoid recomputing them all over again.

1. Create a global empty dictionary within the main function: `mem = new dictionary` or `mem = Init()` (Pseudocode), `mem = {}` (Python)
2. Surround the code of step 1 with the following if query: `if mem.check(k)==false` (Pseudocode), `if k not in mem` (Pseudocode, Python)
3. Replace all return value statements with `mem.add(k, value)` (Pseudocode), `mem[k] = value` (Pseudocode, Python)
4. After the if block, return the appropriate value from the dictionary: `return mem.get(k)` (Pseudocode), `return mem[k]` (Pseudocode, Python)
5. That's it!

```
mem // a global variable
MainAlgo(a)
    mem = Init() // new dictionary
    n = size of a
    return Algo(a, n)

Algo(a, k)
    if k not in mem then                // Same as: mem.check(k)==False
        if k is small then            // Base case
            Compute the return value directly without recursion.
            mem[k] = value              // Same as: mem.add(k, value)
        else
            Make recursive calls on smaller values of k, combine the results, and compute the return
            value
            mem[k] = value
    return mem[k]                      // Same as: return mem.get(k)
```

Step 3: Dynamic Programming Make your solution from step 2 iterative going bottom to top by using dynamic programming.

1. Check what are the minimum and maximum possible values for k . Let us denote them by $k_{\min}$ and $k_{\max}$, respectively. (Typically, but not always, $k_{\min} = 0$ and $k_{\max} = n$.)
2. In the main function, change `mem = Init()` to `mem[kmin .. kmax] = new array filled with 0s`.
3. Afterwards, add a loop that calls the recursive function for all values of k in increasing order:
`for k= kmin to kmax: Algo(a, k)`
4. Change `return Algo(a, n)` to `return mem[kmax]`.
5. In the recursive function, remove the if statement `if k not in mem` (but keep the code that was inside the if)
6. Replace all recursive calls of the form `Algo(k')` by `mem[k']`.
7. Replace everywhere `mem.add(k,value)` by `mem[k] = value`.
8. Remove `return mem.get(k)`.
9. Done! :-)

```
mem // a global variable
MainAlgo(a)
    mem[kmin .. kmax] = new array filled with 0s
    for k = kmin to kmax do
        | Algo(a, k)
    return mem[kmax]

Algo(a, k)
    if k is small then                                     // Base case
        | Compute the return value directly without recursion.
        | mem[k] = value
    else
        | Combine the results for smaller values of k stored in mem and compute the return value
        | mem[k] = value
```

Step 4: Minimize Space You can substantially reduce the space if your computation of $mem[k]$ uses only some last few values of mem . In other words, there has to be some fixed number c such that the computation of $mem[k]$ does not depend on $mem[k_{\min} \dots k-c]$.

For example, if $mem[k] = mem[k-1] + mem[k-2]$, then its computation does not depend on $mem[0 \dots k-3]$. Hence, in that case we have $c = 3$.

Here's how it goes in three simple steps:

1. Check what is c in your algorithm.
2. Change the size of the array mem to c : `mem[0 .. c-1]`.
3. Everywhere change `mem[i]` to `mem[(i)%c]` (for any used value i) where $\%$ is the modulo operator.

That's it!

```
mem // a global variable
MainAlgo(a)
    mem[0 .. c-1] = new array filled with 0s
    for k = kmin to kmax do
        Algo(a, k)
    return mem[kmax%c]

Algo(a, k)
    if k is small then                                     // Base case
        Compute the return value directly without recursion.
        mem[k%c] = value
    else
        Combine the results for smaller values of k (modulo c) stored in mem and compute the
        return value
        mem[k%c] = value
```

Further notes:

There are of course other ways to write memoization and dynamic programming code. Often, one can do it in a more compact way than in the general way presented above. For example, for the dynamic programming, we would use only one function in total without global variables and split the for loop into two loops: one for the base case and the other one for the general case; see below.

```
MainAlgo(a)
    mem[0 .. c-1] = new array filled with 0s
    for all small k do
        Compute the return value directly without recursion.
        mem[k%c] = value
    for all big k do
        Combine the results for smaller values of k (modulo c) stored in mem and compute the
        return value
        mem[k%c] = value
    return mem[k_max%c]
```

- If the return value in the base case is the same for all small k , we can actually fill the array in the beginning with this value. Then we do not need the for loop for the base case anymore.

Note that in the algorithms above, when writing `Algo(a,k)`, we pass only the memory address of the problem a (hence, a pointer to a), and not a copy of the problem a . In Python, this is done automatically: when you call function on a Python list (i.e., array), then actually only the pointer is passed and not the whole array copied.

Our parameter k tells the function, which part of a is relevant; hence, it defines the subproblem.

- One could of course create a subcopy of the original problem a containing exactly the subproblem and pass this subproblem to the recursive call. Then the parameter k would not be needed anymore for the backtracking solution. However, such an approach is very bad: making a separate copy for every recursive call costs, in total, a lot of time and space. Thus, it is much more efficient as what we do above: to pass only a pointer to the original problem together with an integer parameter describing the relevant part for the subproblem.

An efficient alternative to our approach is to declare the whole problem a as a global variable. Then only the parameter k has to be passed to the recursive calls. The asymptotic running time and space complexity remains the same.

For some problems, we define their subproblems with more than one parameter, say t many parameters $k_1, \dots, k_t$. In such cases, you can treat k as a tuple of the form $(k_1, \dots, k_t)$.

- For the base case, by writing `if k is small`, we mean that the values in the tuple are such that we can solve the subproblem directly. For example, it might be the case that we can solve the problem easily if k_1 is 0 independent of all the values of $k_2, \dots, k_t$. Thus, in such a case, the if condition would only check only if k_1 is 0.

The memoization step works perfectly fine for k being a tuple. The dynamic programming step, though, needs a modification: the array mem is now a t -dimensional array of the form $mem[k_{1min} .. k_{1max}] \dots [k_{tmin} .. k_{tmax}]$. Instead of writing `mem[k]`, we now write `mem[k_1] ... [k_t]`. The for loop `for k = k_min to k_max` has to be rewritten as t nested for loops, each one for a different parameter k_i from our tuple k . The order of the for loops might be crucial. Others than that, everything works fine.